

算法设计与分析

第 4 章动态规划笔记

基于 PPT/PDF 的图文整理笔记

原始课件：第 4 章动态规划

整理者：InkTH

日期：2026 年 5 月 29 日

阅读说明

黑色正文表示来自课件的定义、状态设计、状态转移方程、代码与结论的忠实整理；浅蓝色提示框表示为帮助理解而加入的局部说明、例子和提醒。

目录

1 动态规划概述	3
1.1 从斐波那契数列看重复计算	3
1.2 动态规划的基本思想	4
1.3 相关概念与顺序选择	4
1.4 适用条件与基本步骤	6
2 整数拆分问题	6
2.1 问题与状态定义	7
2.2 状态转移方程	7
2.3 自底向上实现	7
2.4 备忘录方法	8
3 最大连续子序列和问题	8
3.1 状态设计	8
3.2 状态转移与求值	9
3.3 复杂度	9
4 最长公共子序列问题	9
4.1 问题与递推规律	9
4.2 求值代码	10
4.3 由 dp 表恢复 LCS	10
4.4 复杂度	11
5 0/1 背包问题	12
5.1 状态定义	12
5.2 状态转移方程	12
5.3 填表与回推	12
5.4 一维优化	13
6 完全背包问题	13
6.1 模型与三重循环解法	14
6.2 改进递推	14
7 资源分配问题	15
7.1 问题建模	15
7.2 状态转移	16

8	会议安排问题	17
8.1	问题与状态设计	17
8.2	状态转移	17
8.3	二分查找与递推实现	17
8.4	复杂度	19
9	本章要点	19

本节知识体系

本节知识体系

本章围绕**如何把一个多阶段决策问题转化成一系列可复用的子问题**展开。主线是**先从斐波那契数列看到重复计算**，再抽象出动态规划的三个关键性质，随后用一组典型问题说明如何设计状态、建立状态转移方程、按顺序填表，并在表中恢复最优解。

- 第一部分说明动态规划的基本思想，包括**最优子结构**、**无后效性**、**重叠子问题**，以及**自底向上填表**和**自顶向下备忘录**两种实现方式。
- 第二部分以**整数拆分**、**最大连续子序列和**、**最长公共子序列**为例，展示计数型、区间/序列型动态规划的状态定义与回推方法。
- 第三部分以**0/1 背包**、**完全背包**、**资源分配**为例，说明选择型动态规划如何在“当前不选”和“当前选若干件”之间递推。
- 第四部分用**会议安排问题**说明：有些问题需要先做排序或二分查找，再把结果接入动态规划，从而把搜索空间压缩为线性或准线性递推。

1 动态规划概述

1.1 从斐波那契数列看重复计算

递归求斐波那契数列时，执行过程是**自顶向下**的。对于同一个较大的 n ，在展开递归树时会多次求解相同的子问题，例如在求 $Fib(5)$ 时， $Fib(3)$ 会被重复计算。

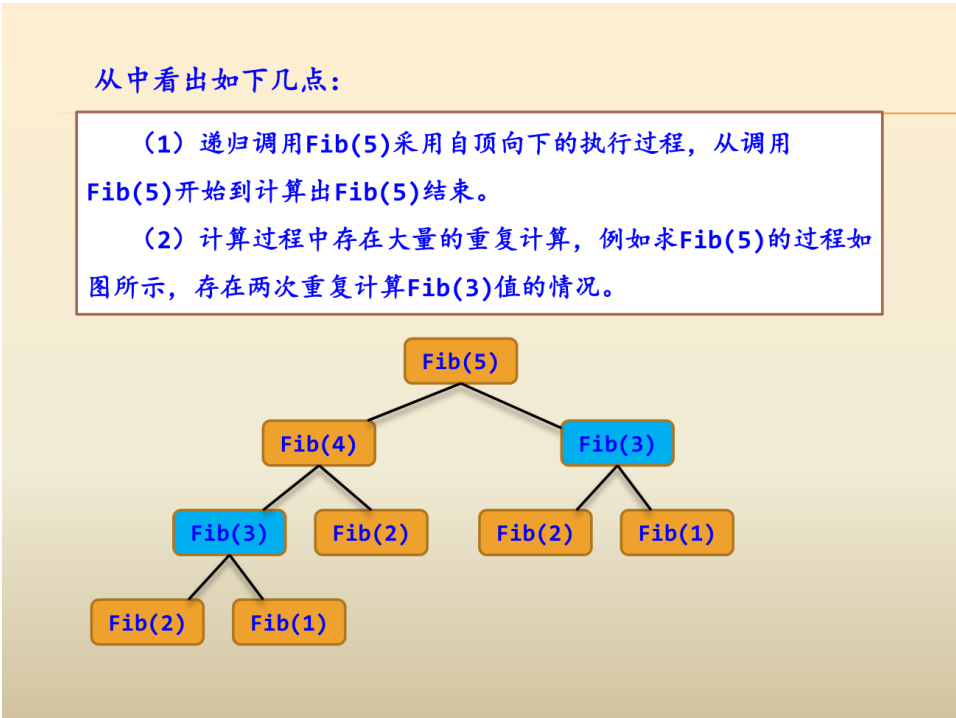


图 1 递归求斐波那契数列时出现的重复计算

为避免重复设计, 可设置动态规划数组 dp , 令 $dp[i]$ 存放 $Fib(i)$ 的值, 先给出边界值 $dp[1] = dp[2] = 1$, 再从 $i = 3$ 到 n 顺序计算。

```
1 int dp[MAX]; // dp[i] 记录 Fib(i) 的值
2
3 int Fib1(int n) {
4     dp[1] = dp[2] = 1; // 边界状态
5     for (int i = 3; i <= n; i++) {
6         dp[i] = dp[i - 1] + dp[i - 2]; // 当前值只依赖前两个子问题
7     }
8     return dp[n];
9 }
```

代码 1 自底向上计算斐波那契数列

1.2 动态规划的基本思想

动态规划是一种解决多阶段决策问题的优化方法。它把原问题转化为一系列单阶段子问题, 利用阶段之间的递推关系, 逐个求解并记录结果。

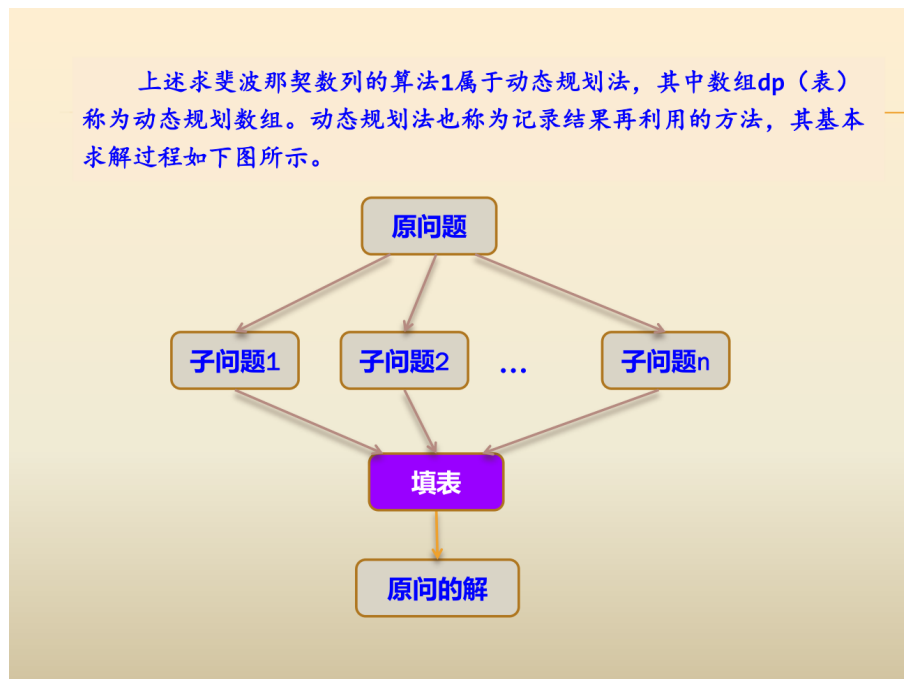


图 2 动态规划“分解子问题、填表、得到原问题解”的基本流程

Bellman-Ford 算法和 Floyd 算法都体现了这一思想:

- Bellman-Ford 把“最多经过 k 条边的最短路”作为阶段;
- Floyd 把“允许经过编号不大于 k 的中间顶点”作为阶段。

1.3 相关概念与顺序选择

在多阶段图模型中, 常用阶段表示决策先后次序, 用状态表示某阶段下问题所处的位置或资源情况。以分阶段最短路径问题为例, 可把从 A 到 E 的决策过程分成 5 个阶段。

在从A~E的过程中，依据按位置所做的决策的次数及所做决策的先后次序，将问题分为5个阶段，阶段变量用于表示各阶段，这里阶段变量 k 为1~5，图中第5阶段是虚拟的一个边界阶段。

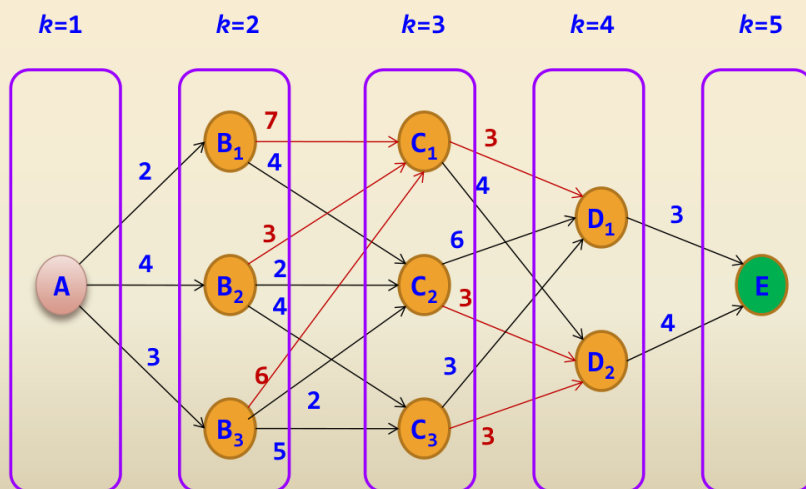


图 3 把分阶段最短路问题拆成多个决策阶段

若用 $f(s)$ 表示状态 s 到终点的最短路径长度，则逆序解法的状态转移可写为

$$f(s_k) = \min_{x_k \in \text{Next}(s_k)} \{c(s_k, x_k) + f(x_k)\}, \quad f(E) = 0.$$

根据求值方向不同，动态规划常见两种做法：

- **逆序解法**：从终点或边界状态向前递推；
- **顺序解法**：从起点状态向后递推。

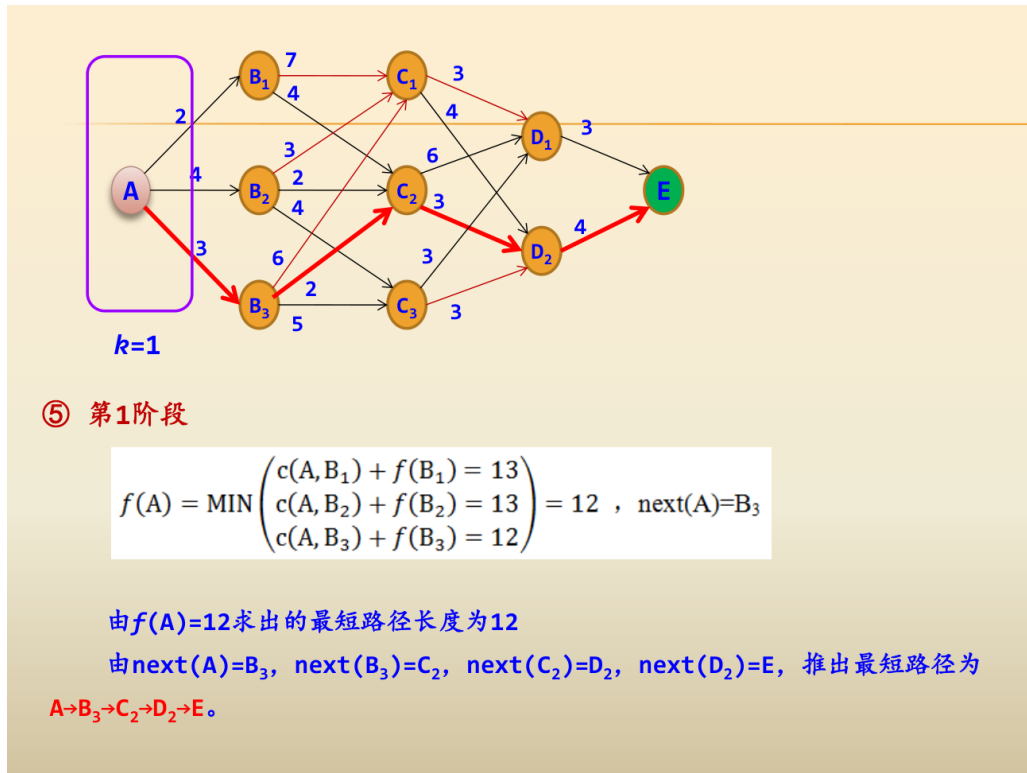


图 4 逆序递推后恢复最短路径的结果

1.4 适用条件与基本步骤

适合用动态规划求解的问题通常具有以下性质：

1. **最优子结构**：原问题的最优解包含其子问题的最优解；
2. **无后效性**：某阶段状态一旦确定，之后的决策不会反过来影响这个状态；
3. **重叠子问题**：不同求解路径中会反复出现相同子问题。

实际求解时，常按以下步骤进行：

1. 分析最优解的性质并刻画结构；
2. 定义状态并写出递归关系；
3. 按自底向上或备忘录方式求最优值；
4. 根据记录信息恢复一个最优解。

Tips

动态规划与分治法都要拆子问题，但分治法更强调子问题相互独立；动态规划则更强调子问题会重复出现，因此值得记忆化或填表。动态规划与贪心法都在逐步做决策，但贪心法一旦决策通常不回退，动态规划会保留所有可能进入最优解的局部结果。

2 整数拆分问题

2.1 问题与状态定义

整数拆分问题要求求将正整数 n 无序拆分成最大数为 k 的拆分方案个数，并且不同方案不重复计数。为避免重复，拆分数按从大到小排序。

令 $f(n, k)$ 表示 n 的 k 拆分方案个数，则有如下分类：

- 当 $n = 1$ 或 $k = 1$ 时，只有 1 种方案；
- 当 $n < k$ 时， $f(n, k) = f(n, n)$ ；
- 当 $n = k$ 时， $f(n, n) = f(n, n - 1) + 1$ ；
- 当 $n > k$ 时，可按“是否包含 k ”分成两类。

2.2 状态转移方程

当 $n > k$ 时：

- 若拆分中包含 k ，剩余部分是 $(n - k)$ 的 k 拆分，方案数为 $f(n - k, k)$ ；
- 若拆分中不包含 k ，则所有拆分数都小于 k ，方案数为 $f(n, k - 1)$ 。

因此得到状态转移方程

$$f(n, k) = \begin{cases} 1, & n = 1 \text{ 或 } k = 1, \\ f(n, n), & n < k, \\ f(n, n - 1) + 1, & n = k, \\ f(n - k, k) + f(n, k - 1), & n > k. \end{cases}$$

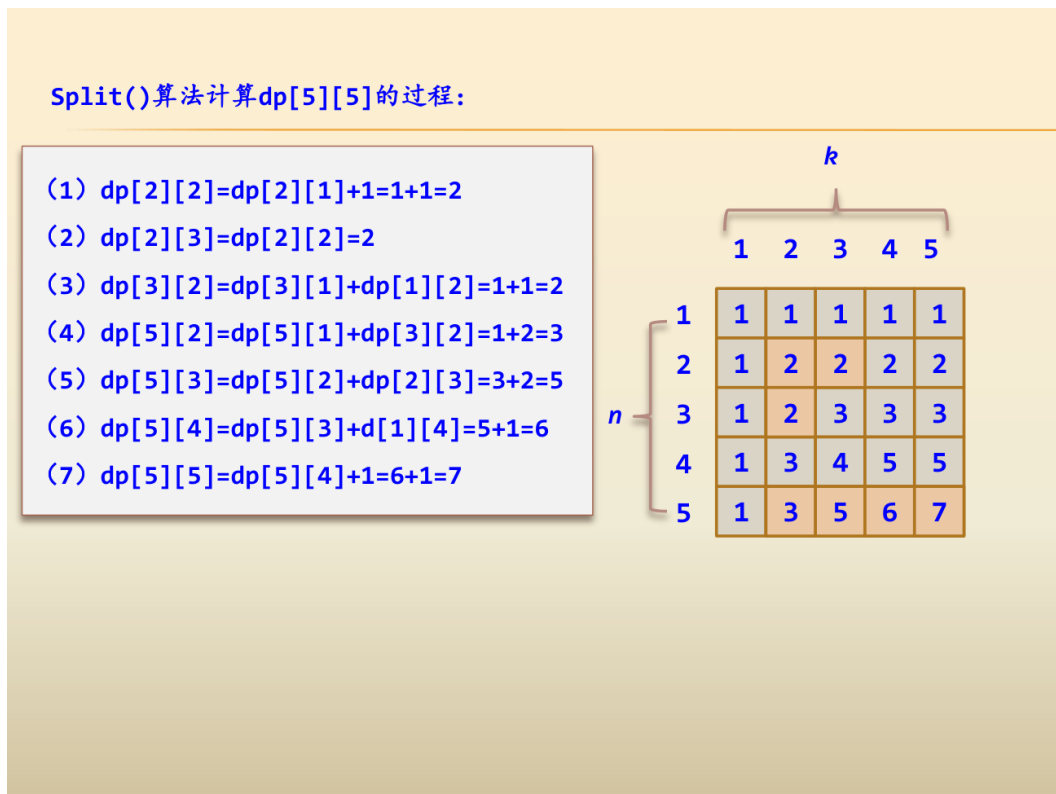
2.3 自底向上实现

设置二维动态规划数组 dp ，用 $dp[n][k]$ 存放 $f(n, k)$ 。

```

1 int dp[MAXN][MAXN];
2
3 void Split(int n, int k) {
4     for (int i = 1; i <= n; i++) {
5         for (int j = 1; j <= k; j++) {
6             if (i == 1 || j == 1)
7                 dp[i][j] = 1; // 只有一种拆法
8             else if (i < j)
9                 dp[i][j] = dp[i][i]; // 最大拆分数不能超过 i
10            else if (i == j)
11                dp[i][j] = dp[i][j - 1] + 1; // 增加“单独一个 i”这类方案
12            else
13                dp[i][j] = dp[i][j - 1] + dp[i - j][j];
14        }
15    }
16 }
```

代码 2 整数拆分的自底向上动态规划

图 5 计算 $dp[5][5]$ 时的关键递推过程与结果表

2.4 备忘录方法

该问题本身可直接写成递归，但由于子问题重叠，会出现大量重复计算。为此可以在递归中增加数组 dp 作为备忘录：若某个 $dp[n][k]$ 已经算出，就直接返回，不再重复展开。

Explanation

自底向上与备忘录方法使用的是同一组状态和同一条递推式。区别只在于：前者先规划好求值顺序并显式填整张表，后者按递归需要“按需计算”，算过的状态才会写入表中。

3 最大连续子序列和问题

3.1 状态设计

给定整数序列 $a_1, a_2, \dots, a_n$ ，要求求出最大连续子序列和，并规定结果至少为 0。

令

$$b_j = \max\{a_i + a_{i+1} + \dots + a_j \mid 1 \leq i \leq j\},$$

表示以前 j 个元素中、且以 a_j 作为结尾的最大连续子序列和。于是有：

- 若 $b_{j-1} > 0$ ，则继续把 a_j 接在前一个最优后缀后面；
- 若 $b_{j-1} \leq 0$ ，则前面的贡献不再保留，从 a_j 重新开始。

3.2 状态转移与求值

设置一维数组 dp 存放 b_j , 边界为 $dp[0] = 0$, 转移方程为

$$dp[j] = \max\{dp[j-1] + a_j, a_j\}, \quad 1 \leq j \leq n.$$

整个序列的最大连续子序列和就是所有 $dp[j]$ 中的最大者。

```

1 int a[] = {0, -2, 11, -4, 13, -5, -2}; // 从下标 1 开始使用
2 int dp[MAXN];
3
4 void maxSubSum(int n) {
5     dp[0] = 0; // 空前缀的边界值
6     for (int j = 1; j <= n; j++) {
7         dp[j] = max(dp[j-1] + a[j], a[j]); // 保留前缀或从当前位置重启
8     }
9 }

```

代码 3 最大连续子序列和的线性动态规划

若要恢复具体子序列, 可先找到 dp 中的最大值位置 $maxj$, 再向前回溯到最近一个 $dp[k] \leq 0$ 的位置, 区间 $(k, maxj]$ 即为所求子序列。

3.3 复杂度

该算法只需一次线性扫描, 因此时间复杂度为

$$O(n).$$

4 最长公共子序列问题

4.1 问题与递推规律

若序列 Z 同时是序列 A 和 B 的子序列, 则称 Z 是 A 和 B 的公共子序列。最长公共子序列问题要求求出长度最大的公共子序列。

设 $A = (a_0, a_1, \dots, a_{m-1})$, $B = (b_0, b_1, \dots, b_{n-1})$ 。定义二维数组 dp , 其中

$$dp[i][j]$$

表示子序列 $(a_0, \dots, a_{i-1})$ 与 $(b_0, \dots, b_{j-1})$ 的最长公共子序列长度。

根据最后一个字符是否相同, 可分两种情况:

- 若 $a_{i-1} = b_{j-1}$, 则 $dp[i][j] = dp[i-1][j-1] + 1$;
- 若 $a_{i-1} \neq b_{j-1}$, 则 $dp[i][j] = \max\{dp[i][j-1], dp[i-1][j]\}$ 。

边界条件为:

$$dp[i][0] = 0, \quad dp[0][j] = 0.$$

4.2 求值代码

```

1 void LCSlength(string a, string b, int m, int n) {
2     for (int i = 0; i <= m; i++)
3         dp[i][0] = 0; // 任意序列与空串的 LCS 长度为 0
4     for (int j = 0; j <= n; j++)
5         dp[0][j] = 0;
6
7     for (int i = 1; i <= m; i++) {
8         for (int j = 1; j <= n; j++) {
9             if (a[i - 1] == b[j - 1])
10                dp[i][j] = dp[i - 1][j - 1] + 1;
11            else
12                dp[i][j] = max(dp[i][j - 1], dp[i - 1][j]);
13        }
14    }
15 }

```

代码 4 LCS 长度的动态规划计算

4.3 由 dp 表恢复 LCS

从 $dp[m][n]$ 开始回推：

- 若 $dp[i][j] = dp[i - 1][j]$ ，则 $i--$ ；
- 否则若 $dp[i][j] = dp[i][j - 1]$ ，则 $j--$ ；
- 否则说明 $a_{i-1} = b_{j-1}$ 且该字符属于某个 LCS，把它加入答案，再令 $i--, j--$ 。

```

1 void Buildsups(string a, int m, int n, vector<char>& subs) {
2     int k = dp[m][n]; // 还需要回收的字符个数
3     int i = m, j = n;
4     while (k > 0) {
5         if (dp[i][j] == dp[i - 1][j])
6             i--; // 当前最优值来自上方
7         else if (dp[i][j] == dp[i][j - 1])
8             j--; // 当前最优值来自左方
9         else {
10             subs.push_back(a[i - 1]); // 左、上都不同，说明该字符被选中
11             i--;
12             j--;
13             k--;
14         }
15     }
16 }

```

代码 5 根据 dp 表回推最长公共子序列

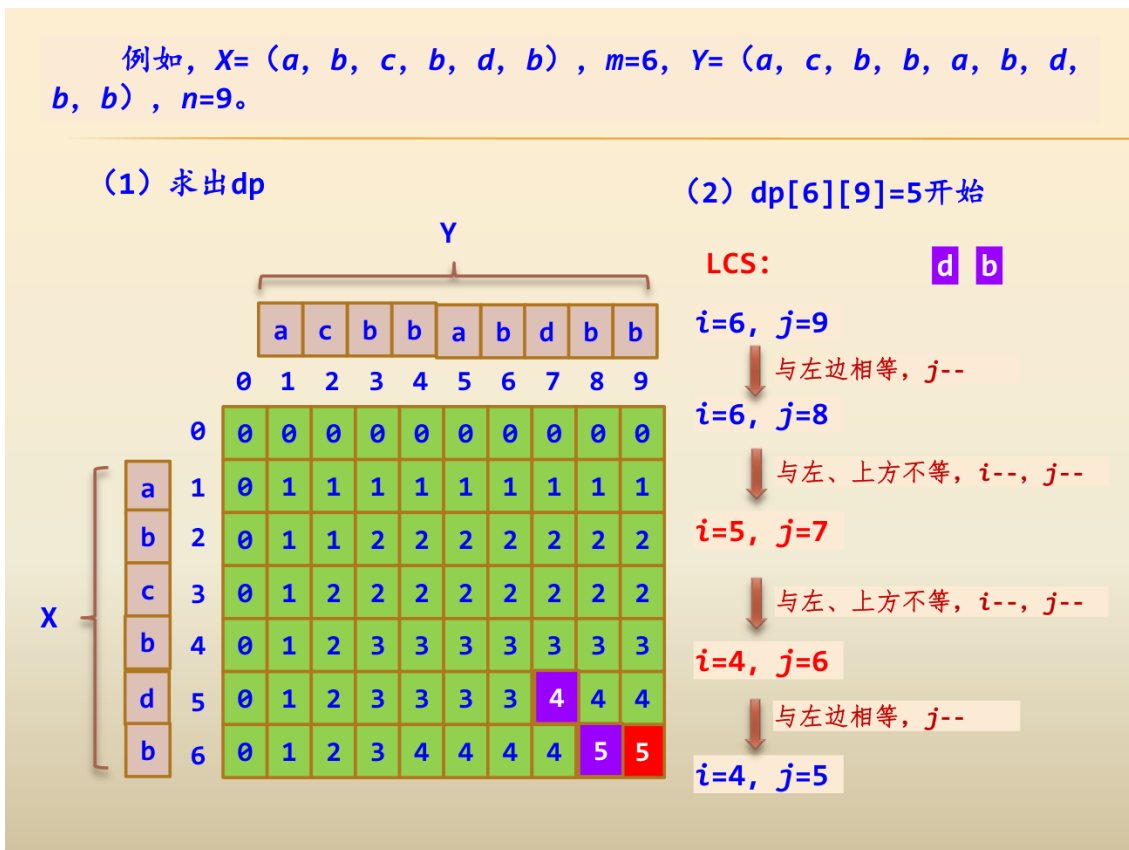


图 6 LCS 例子中 dp 表的构造与回推路径

Example

题目. 设 $X = (a, b, c, b, d, b)$, $Y = (a, c, b, b, a, b, d, b, b)$, 求它们的最长公共子序列。

思路. 先按前缀长度构造二维 dp 表, 再从 $dp[6][9]$ 开始逆向回推。

推导.

1. 计算出 $dp[6][9] = 5$, 说明 LCS 长度为 5;
2. 回推时, 若当前位置与左边相等则左移, 若与上方相等则上移;
3. 若当前位置既不等于左边也不等于上方, 则说明当前字符被选入 LCS;
4. 按回推顺序得到的字符是反向的, 最后再逆序输出。

$$dp[6][9] = 5 \Rightarrow \text{LCS 长度为 5}$$

答案. 该例可得到一个最长公共子序列为 **acbdb**。

4.4 复杂度

由于使用了两重循环, 求 LCS 长度的时间复杂度和空间复杂度均为

$$O(mn).$$

5 0/1 背包问题

5.1 状态定义

有 n 个物品，重量分别为 $w_1, w_2, \dots, w_n$ ，价值分别为 $v_1, v_2, \dots, v_n$ ，背包容量为 W 。每个物品要么选 0 次，要么选 1 次。

定义二维数组 dp ，其中

$$dp[i][r]$$

表示背包容量为 r ，且只考虑前 i 个物品时可获得的最优价值。

5.2 状态转移方程

边界条件为

$$dp[i][0] = 0, \quad dp[0][r] = 0.$$

若第 i 个物品放不下，即 $r < w[i]$ ，则

$$dp[i][r] = dp[i-1][r].$$

否则在“不放第 i 个物品”和“放第 i 个物品”之间取较优：

$$dp[i][r] = \max\{dp[i-1][r], dp[i-1][r-w[i]] + v[i]\}.$$

5.3 填表与回推

```

1 void Knap(int n, int W) {
2     for (int i = 0; i <= n; i++)
3         dp[i][0] = 0;
4     for (int r = 0; r <= W; r++)
5         dp[0][r] = 0;
6
7     for (int i = 1; i <= n; i++) {
8         for (int r = 1; r <= W; r++) {
9             if (r < w[i])
10                dp[i][r] = dp[i-1][r];    // 放不下，只能继承上一行
11            else
12                dp[i][r] = max(dp[i-1][r],
13                             dp[i-1][r-w[i]] + v[i]);
14        }
15    }
16 }
```

代码 6 0/1 背包的二维动态规划

从 $dp[n][W]$ 开始回推，若 $dp[i][r] \neq dp[i-1][r]$ ，说明选择了物品 i ，并令 $r = r - w[i]$ ；否则说明未选择该物品。

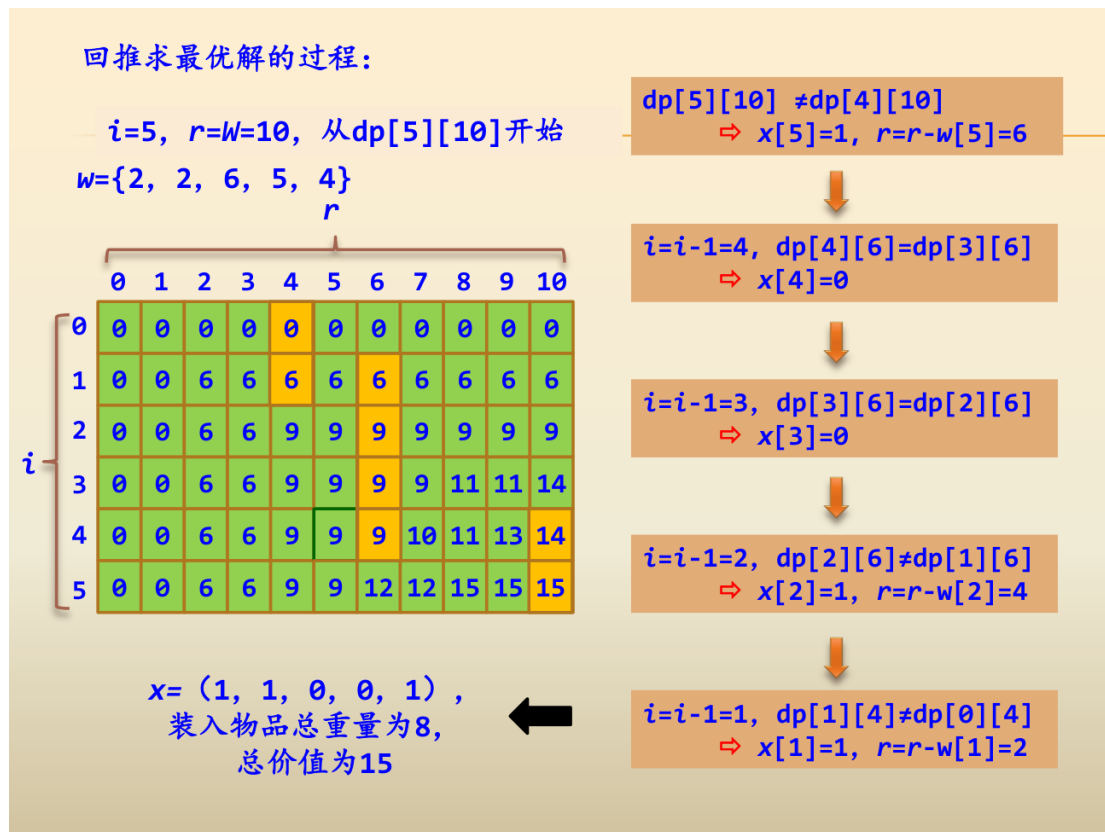


图 7 0/1 背包例子中 dp 表与最优解回推过程

5.4 一维优化

二维状态的空间可进一步压缩成一维。关键是容量 r 必须从大到小遍历，以确保当前物品只被使用一次。

```

1 void Knap2(int n, int W) {
2     for (int r = 0; r <= W; r++)
3         dp[r] = 0;
4
5     for (int i = 1; i <= n; i++) {
6         for (int r = W; r >= 1; r--) { // 逆序保证每件物品最多使用一次
7             if (r >= w[i])
8                 dp[r] = max(dp[r], dp[r - w[i]] + v[i]);
9         }
10    }
11 }

```

代码 7 0/1 背包的一维空间优化

Note

时间复杂度 $O(nW)$ 依赖于容量 W 的数值大小，而不是输入长度的对数，因此这是一个伪多项式时间算法。

6 完全背包问题

6.1 模型与三重循环解法

完全背包与 0/1 背包的区别在于：每种物品可以被选择任意多件。

定义 $dp[i][j]$ 为前 i 种物品在容量不超过 j 时的最大价值，同时用 $fk[i][j]$ 记录第 i 种物品取了多少件。朴素状态转移为

$$dp[i][j] = \max_{kw[i] \leq j} \{dp[i-1][j - kw[i]] + kv[i]\}.$$

这会产生三重循环，时间复杂度为 $O(nW^2)$ 。

6.2 改进递推

由于“选 k 件物品 i ”可以由“在 $dp[i][j - w[i]]$ 的基础上再多选 1 件物品 i ”得到，因此可把递推改写为：

$$dp[i][j] = \begin{cases} dp[i-1][j], & j < w[i], \\ \max\{dp[i-1][j], dp[i][j - w[i]] + v[i]\}, & j \geq w[i]. \end{cases}$$

这里与 0/1 背包的关键差别在于第二项使用的是**同一行**的 $dp[i][j - w[i]]$ ，表示当前物品可以重复选。

```

1 int solve1(int n, int W) {
2     for (int i = 1; i <= n; i++) {
3         for (int j = 0; j <= W; j++) {
4             if (j < w[i])
5                 dp[i][j] = dp[i-1][j];
6             else
7                 dp[i][j] = max(dp[i-1][j], dp[i][j - w[i]] + v[i]);
8         }
9     }
10    return dp[n][W];
11 }

```

代码 8 完全背包的二维动态规划

例如, $n=3$, $W=7$, $w=(3, 4, 2)$, $v=(4, 5, 3)$ 时, 其求解结果如下表所示, 表中元素为 $dp(i, j)[fk(i, j)]$, $dp(n, W)$ 为最终结果, 即最大价值总和为 10。

$i \backslash j$	0	1	2	3	4	5	6	7
0	0[0]	0[0]	0[0]	0[0]	0[0]	0[0]	0[0]	0[0]
1	0[0]	0[0]	0[0]	4[1]	4[1]	4[1]	8[2]	8[2]
2	0[0]	0[0]	0[0]	4[0]	5[1]	5[1]	8[0]	9[1]
3	0[0]	0[0]	3[1]	4[0]	6[1]	7[1]	9[3]	10[2]

回推过程:

- (1) $i=3$; $dp[3][7]=10$, $fk[3][7]=2$, 物品3挑选2件
- (2) $i=2$; $dp[2][W-2 \times 2]=dp[2][3]=4$, $fk[2][3]=0$, 物品2挑选0件
- (3) $i=1$; $dp[1][3]=4$, $fk[1][3]=1$, 物品1挑选1件。

图 8 完全背包例子中的 dp/fk 表与回推结果

7 资源分配问题

7.1 问题建模

资源分配问题要求把数量一定的资源分配给若干使用者, 使总收益最大。例子中总员工数为 $n=5$, 商店数为 $m=3$, 收益表如下。

4.7 求解资源分配问题

【问题描述】 资源分配问题是将数量一定的一种或若干种资源（原材料、资金、设备或劳动力等），合理地分配给若干使用者，使总收益最大。

例如，某公司有3个商店A、B、C，拟将新招聘的5名员工分配给这3个商店，各商店得到新员工后，每年的赢利情况如下表所示，求分配给各商店各多少员工才能使公司的赢利最大？

商店 \ 员工数	0人	1人	2人	3人	4人	5人
A	0	3	7	9	12	13
B	0	5	10	11	11	11
C	0	4	6	11	12	12

图 9 三家商店在不同员工数下的收益表

若从商店 m 逆序决策，定义

$$dp[i][s]$$

表示考虑商店 i 到商店 m ，并且总共还可分配 s 个人时的最优收益。再设置 $pnum[i][s]$ 记录商店 i 被分配的人数。

7.2 状态转移

边界为

$$dp[m+1][j] = 0.$$

对每个商店 i 和剩余人数 s ，枚举给商店 i 分配 j 个人，则有

$$dp[i][s] = \max_{0 \leq j \leq s} \{v[i][j] + dp[i+1][s-j]\}.$$

当某个 j 取到最大值时，记录 $pnum[i][s] = j$ 。

```

1 void Plan(int m, int n) {
2     for (int j = 0; j <= n; j++)
3         dp[m+1][j] = 0; // 越过最后一个商店后的收益为 0
4
5     for (int i = m; i >= 1; i--) {
6         for (int s = 1; s <= n; s++) {
7             int maxf = 0, maxj = 0;
8             for (int j = 0; j <= s; j++) { // 枚举当前商店分配 j 人
9                 if (v[i][j] + dp[i+1][s-j] >= maxf) {
10                     maxf = v[i][j] + dp[i+1][s-j];
11                 }
12             }
13             pnum[i][s] = maxj;
14         }
15     }
16 }
```

```

11         maxj = j;
12     }
13 }
14 dp[i][s] = maxf;
15 pnum[i][s] = maxj;
16 }
17 }
18 }

```

代码 9 资源分配问题的动态规划

8 会议安排问题

8.1 问题与状态设计

会议安排问题只有一个教室，任意两个时间重叠的订单不能同时接受。目标是选择一组互相兼容的订单，使总活动时间最大。

先按订单结束时间递增排序。设 $A[i]$ 为第 i 个订单， $\text{length}_i = A[i].e - A[i].b$ 。定义

$$dp[i]$$

表示在订单 $A[0..i]$ 中能取得的最大兼容总时长。

8.2 状态转移

对当前订单 $A[i]$ ，找到一个最晚结束且不晚于 $A[i]$ 开始时间的订单 $A[j]$ 。则

$$dp[i] = \max\{dp[i-1], dp[j] + \text{length}_i\}.$$

若不存在这样的 j ，则将第二项改为 length_i 。

另外设置数组 pre ：

- $pre[i] = -1$ ：选中了订单 i ，且它前面没有兼容订单；
- $pre[i] = -2$ ：当前最优解没有选订单 i ；
- $pre[i] = j$ ：选中了订单 i ，其前驱是订单 j 。

8.3 二分查找与递推实现

由于订单已按结束时间排序，可用二分查找在 $A[0..i-1]$ 中寻找最后一个满足 $A[j].e \leq A[i].b$ 的订单。

```

1 void solve() {
2     memset(dp, 0, sizeof(dp));
3     stable_sort(A, A + n);           // 按结束时间递增排序
4     dp[0] = A[0].length;
5     pre[0] = -1;
6
7     for (int i = 1; i < n; i++) {

```

```
8      int low = 0, high = i - 1;
9      while (low <= high) {                // 二分找最晚兼容订单
10         int mid = (low + high) / 2;
11         if (A[mid].e <= A[i].b)
12             low = mid + 1;
13         else
14             high = mid - 1;
15     }
16
17     if (low == 0) {
18         if (dp[i - 1] >= A[i].length) {
19             dp[i] = dp[i - 1];
20             pre[i] = -2;                    // 不选择当前订单
21         } else {
22             dp[i] = A[i].length;
23             pre[i] = -1;                    // 选择当前订单且没有前驱
24         }
25     } else {
26         if (dp[i - 1] >= dp[low - 1] + A[i].length) {
27             dp[i] = dp[i - 1];
28             pre[i] = -2;
29         } else {
30             dp[i] = dp[low - 1] + A[i].length;
31             pre[i] = low - 1;
32         }
33     }
34 }
35 }
```

代码 10 会议安排问题中的排序、二分与动态规划

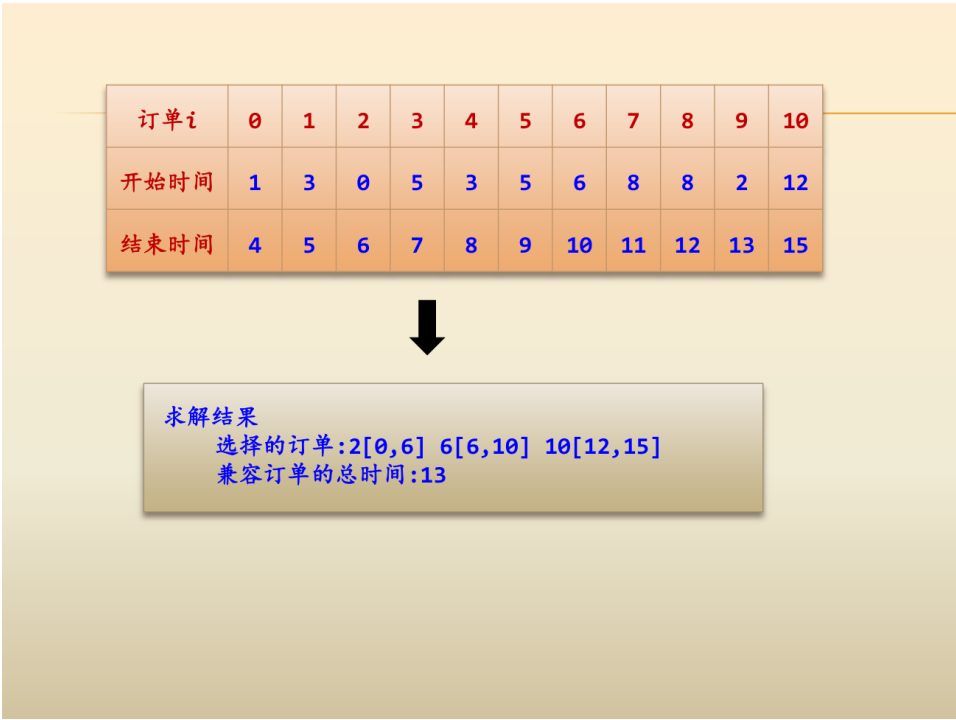


图 10 会议安排问题示例的最终选择结果

8.4 复杂度

外层循环进行 n 次，每次用二分查找寻找前驱订单，因此总时间复杂度为

$$O(n \log n).$$

9 本章要点

Summary

- 动态规划的关键不是“会不会写表”，而是**能否找到合适的状态表示与递推关系**。
- 最优子结构与无后效性是核心前提；重叠子问题决定了填表或记忆化是否能显著节省计算。
- 计数型问题常把“方案是否包含某元素”分情况讨论；选择型问题常在“当前不选”和“当前选”之间取最优；序列型问题则常围绕“以前缀结尾”或“两个前缀”设计状态。
- 除了求最优值，还要记录足够的信息用于恢复最优解，例如 $next/pre$ 、 x 、 fk 、 $pnum$ 等辅助数组。
- 0/1 背包与完全背包的区别，集中体现在状态转移用上一行还是当前行，以及一维优化时容量循环方向是否逆序。